

Nama : Muhammad Aksan

Kelas : RPL - B

Nim : 105841107223

## 1.1 Latar Belakang

Seiring meningkatnya kebutuhan terhadap sistem basis data yang andal, cepat, dan tersedia secara terus-menerus (high availability), salah satu teknik yang banyak digunakan adalah Database Replication. Replikasi memungkinkan data dari satu server basis data (source/master) disalin secara otomatis ke satu atau lebih server lainnya (replica/slave), sehingga jika satu server mengalami kegagalan, sistem masih bisa berjalan dari server lain.

Praktikum ini bertujuan untuk memahami dan mengimplementasikan tiga skenario replikasi basis data: (1) Master-Slave, (2) Master-Master, dan (3) perbandingan antara mode Asynchronous dan Semi-Synchronous replication, menggunakan MySQL 8.0 yang dijalankan di atas Docker.

## 1.2 Tujuan

- ☐ Memahami konsep dan cara kerja Database Replication.
- ☐ Mengimplementasikan topologi Master-Slave secara manual.
- ☐ Mengimplementasikan topologi Master-Master dengan pencegahan konflik primary key.
- ☐ Membandingkan perilaku replication mode Asynchronous dan Semi-Synchronous.
- ☐ Melakukan verifikasi hasil replikasi melalui query langsung ke masing-masing node.

## 1.3 Alat dan Bahan

| Komponen        | Keterangan                                  |
|-----------------|---------------------------------------------|
| Sistem Operasi  | Ubuntu 22.04 LTS (Linux)                    |
| Docker          | Docker Engine v26.x + Docker Compose v2.x   |
| Database Engine | MySQL 8.0 (image resmi dari Docker Hub)     |
| Tools Query     | MySQL CLI (mysql client di dalam container) |
| Editor          | Visual Studio Code                          |

## 2.1 Database Replication

Database Replication adalah proses menyalin dan mempertahankan objek basis data, seperti tabel dan skema, di beberapa server basis data secara bersamaan. Tujuan

utamanya meliputi peningkatan ketersediaan (availability), skalabilitas baca (read scalability), toleransi kesalahan (fault tolerance), dan pencadangan data (backup).

Pada MySQL, mekanisme replikasi bekerja berdasarkan Binary Log (binlog). Master mencatat setiap perubahan data ke dalam binlog, kemudian slave membaca dan menerapkan perubahan tersebut secara lokal melalui proses I/O thread dan SQL thread.

## 2.2 Topologi Master-Slave

Pada topologi Master-Slave, hanya satu server (master) yang menerima operasi tulis (INSERT, UPDATE, DELETE). Server lainnya (slave) hanya melayani operasi baca (SELECT). Konfigurasi utama: master mengaktifkan `log_bin`, slave dikonfigurasi `read_only=ON` dan menghubungkan diri ke master menggunakan `CHANGE REPLICATION SOURCE TO`.

## 2.3 Topologi Master-Master

Pada topologi Master-Master (atau Multi-Primary), kedua node berfungsi sekaligus sebagai master dan slave. Keduanya dapat menerima operasi tulis. Untuk menghindari konflik primary key, digunakan parameter `auto_increment_increment` dan `auto_increment_offset` yang berbeda pada tiap node, sehingga ID yang dihasilkan tidak pernah bertabrakan.

## 2.4 Asynchronous vs Semi-Synchronous Replication

| Aspek         | Asynchronous                                        | Semi-Synchronous                                         |
|---------------|-----------------------------------------------------|----------------------------------------------------------|
| Cara Kerja    | Master commit langsung tanpa menunggu slave         | Master menunggu ACK dari $\geq 1$ slave sebelum commit   |
| Kecepatan     | Sangat cepat                                        | Sedikit lebih lambat (ada overhead jaringan)             |
| Keamanan Data | Bisa kehilangan data jika master crash sebelum sync | Lebih aman; data pasti sudah di slave saat commit sukses |

## 3.1 Skenario 1: Master-Slave Replication

### 3.1.1 Konfigurasi dan Menjalankan Container

Langkah pertama adalah masuk ke direktori master-slave dan menjalankan container menggunakan Docker Compose. Container master akan menginisialisasi database demo\_db beserta tabel dan data awal secara otomatis dari script SQL yang telah disiapkan.

*Perintah yang dijalankan:*

```
$ cd ~/Desktop/db-replication/master-slave
$ docker compose up -d

[+] Running 5/5
✔ Network master-slave_ms-network Created      0.1s
✔ Volume "master-slave_ms-master-data" Created 0.0s
✔ Volume "master-slave_ms-slave-data" Created  0.0s
✔ Container ms-master      Started              1.2s
✔ Container ms-slave       Started              2.8s
```

Kedua container berhasil berjalan. Container ms-slave menunggu ms-master sehat (healthcheck) sebelum mulai, sesuai dengan konfigurasi depends\_on di docker-compose.yml.

### 3.1.2 Verifikasi Container Berjalan

```
$ docker ps
```

| CONTAINER ID | IMAGE     | COMMAND                  | STATUS                  | PORTS                  | NAMES     |
|--------------|-----------|--------------------------|-------------------------|------------------------|-----------|
| a3f9c2e10d41 | mysql:8.0 | "docker-entrypoint.s..." | Up 32 seconds (healthy) | 0.0.0.0:3306->3306/tcp | ms-master |
| 7b81de42fc09 | mysql:8.0 | "docker-entrypoint.s..." | Up 18 seconds           | 0.0.0.0:3307->3306/tcp | ms-slave  |

### 3.1.3 Melihat Status Binary Log di Master

Sebelum mengonfigurasi slave, saya perlu mendapatkan posisi Binary Log yang sedang aktif di master. Informasi ini (nama file dan posisi) diperlukan agar slave tahu dari titik mana ia harus mulai membaca perubahan.

```
$ docker exec -it ms-master mysql -u root -prootpass

mysql> SHOW MASTER STATUS;
+-----+-----+-----+-----+-----+
| File           | Position | Binlog_Do_DB | Binlog_Ignore_DB | Executed_Gtid_Set |
+-----+-----+-----+-----+-----+
| mysql-bin.000003 |      897 | demo_db      |                   |                   |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Catat: File = mysql-bin.000003, Position = 897. Angka ini digunakan di langkah berikutnya.

### 3.1.4 Konfigurasi Slave

Selanjutnya saya masuk ke container slave dan menjalankan perintah CHANGE REPLICATION SOURCE TO untuk menghubungkan slave ke master menggunakan posisi binlog yang telah dicatat.

```
$ docker exec -it ms-slave mysql -u root -prootpass
```

```
mysql> CHANGE REPLICATION SOURCE TO
-> SOURCE_HOST = 'ms-master',
-> SOURCE_USER = 'replicator',
-> SOURCE_PASSWORD = 'replicapass',
-> SOURCE_LOG_FILE = 'mysql-bin.000003',
-> SOURCE_LOG_POS = 897;
Query OK, 0 rows affected, 2 warnings (0.03 sec)
```

```
mysql> START REPLICA;
Query OK, 0 rows affected (0.02 sec)
```

### 3.1.5 Cek Status Replication

```
mysql> SHOW REPLICA STATUS\G
```

```
***** 1. row *****
Replica_IO_State: Waiting for source to send event
Source_Host: ms-master
Source_User: replicator
Source_Port: 3306
Source_Log_File: mysql-bin.000003
Read_Source_Log_Pos: 897
Relay_Log_File: relay-bin.000002
Relay_Log_Pos: 329
Relay_Source_Log_File: mysql-bin.000003
Replica_IO_Running: Yes
Replica_SQL_Running: Yes
Seconds_Behind_Source: 0
1 row in set (0.00 sec)
```

Replica\_IO\_Running: Yes dan Replica\_SQL\_Running: Yes — Replication berjalan normal.  
Lag = 0 detik.

### 3.1.6 Pengujian: Insert Data di Master

Saya melakukan INSERT di master untuk membuktikan bahwa perubahan data akan otomatis ter-replikasi ke slave.

```
-- Di ms-master
mysql> USE demo_db;
Database changed

mysql> INSERT INTO users (name, email) VALUES ('Dave', 'dave@demo.com');
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO users (name, email) VALUES ('Eva', 'eva@demo.com');
Query OK, 1 row affected (0.01 sec)
```

```
mysql> UPDATE products SET stock = stock - 2 WHERE name = 'Laptop';
Query OK, 1 row affected (0.01 sec)
```

```
mysql> SELECT * FROM users;
+----+-----+-----+-----+
| id | name  | email      | created_at |
+----+-----+-----+-----+
| 1  | Alice | alice@demo.com | 2026-07-14 09:15:22 |
| 2  | Bob   | bob@demo.com   | 2026-07-14 09:15:22 |
| 3  | Charlie | charlie@demo.com | 2026-07-14 09:15:22 |
| 4  | Dave  | dave@demo.com  | 2026-07-14 09:32:17 |
| 5  | Eva   | eva@demo.com   | 2026-07-14 09:32:19 |
+----+-----+-----+-----+
5 rows in set (0.00 sec)
```

### 3.1.7 Verifikasi Data di Slave

```
-- Di ms-slave
mysql> USE demo_db;
Database changed

mysql> SELECT * FROM users;
+----+-----+-----+-----+
| id | name  | email      | created_at |
+----+-----+-----+-----+
| 1  | Alice | alice@demo.com | 2026-07-14 09:15:22 |
| 2  | Bob   | bob@demo.com   | 2026-07-14 09:15:22 |
| 3  | Charlie | charlie@demo.com | 2026-07-14 09:15:22 |
| 4  | Dave  | dave@demo.com  | 2026-07-14 09:32:17 |
| 5  | Eva   | eva@demo.com   | 2026-07-14 09:32:19 |
+----+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM products;
+----+-----+-----+-----+
| id | name   | price  | stock | updated_at |
+----+-----+-----+-----+
| 1  | Laptop | 950000.00 | 8 | 2026-07-14 09:32:20 |
| 2  | Mouse  | 150000.00 | 50 | 2026-07-14 09:15:22 |
| 3  | Keyboard | 250000.00 | 30 | 2026-07-14 09:15:22 |
+----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Semua data dari master berhasil ter-replikasi ke slave. Stock Laptop berkurang dari 10 menjadi 8 sesuai UPDATE di master.

### 3.1.8 Uji Proteksi Read-Only di Slave

Saya mencoba melakukan operasi tulis langsung ke slave untuk membuktikan bahwa slave dilindungi oleh konfigurasi `super_read_only = ON`.

```
-- Di ms-slave
mysql> INSERT INTO users (name, email) VALUES ('Hacker', 'hacker@test.com');
ERROR 1290 (HY000): The MySQL server is running with the --super-read-only option
so it cannot execute this statement
```

Slave berhasil menolak operasi tulis. Konfigurasi read\_only berfungsi dengan baik.

## 3.2 Skenario 2: Master-Master Replication

### 3.2.1 Menjalankan Container

```
$ cd ~/Desktop/db-replication/master-master
$ docker compose up -d
```

[+] Running 5/5

```
✓ Network master-master_mm-network Created    0.1s
✓ Volume "master-master_mm-node1-data" Created 0.0s
✓ Volume "master-master_mm-node2-data" Created 0.0s
✓ Container mm-node1      Started    1.4s
✓ Container mm-node2      Started    3.1s
```

### 3.2.2 Setup: Node2 Replikasi dari Node1

Pertama saya ambil posisi binlog Node1, lalu konfigurasi Node2 agar mereplikasi dari Node1.

```
$ docker exec -it mm-node1 mysql -u root -prootpass
```

```
mysql> SHOW MASTER STATUS;
+-----+-----+-----+-----+
| File          | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+-----+-----+-----+-----+
| mysql-bin.000003 | 1204 |          |          |
+-----+-----+-----+-----+
```

```
$ docker exec -it mm-node2 mysql -u root -prootpass
```

```
mysql> CHANGE REPLICATION SOURCE TO
-> SOURCE_HOST = 'mm-node1',
-> SOURCE_USER = 'replicator',
-> SOURCE_PASSWORD = 'replicapass',
-> SOURCE_LOG_FILE = 'mysql-bin.000003',
-> SOURCE_LOG_POS = 1204;
```

Query OK, 0 rows affected (0.02 sec)

```
mysql> START REPLICA;
```

Query OK, 0 rows affected (0.02 sec)

### 3.2.3 Setup: Node1 Replikasi dari Node2

```
-- Ambil posisi binlog Node2 (masih di terminal Node2)
```

```
mysql> SHOW MASTER STATUS;
```

```
+-----+-----+
| File          | Position |
+-----+-----+
```

```
+-----+-----+
| mysql-bin.000003 | 489 |
+-----+-----+
```

```
$ docker exec -it mm-node1 mysql -u root -prootpass
```

```
mysql> CHANGE REPLICATION SOURCE TO
-> SOURCE_HOST = 'mm-node2',
-> SOURCE_USER = 'replicator',
-> SOURCE_PASSWORD = 'replicapass',
-> SOURCE_LOG_FILE = 'mysql-bin.000003',
-> SOURCE_LOG_POS = 489;
```

```
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> START REPLICA;
```

```
Query OK, 0 rows affected (0.01 sec)
```

Kedua node kini saling mereplikasi satu sama lain (bidirectional replication aktif).

### 3.2.4 Pengujian Write di Node1

```
-- Di mm-node1
```

```
mysql> USE demo_db;
```

```
mysql> INSERT INTO orders (customer, product, amount, node_src) VALUES
```

```
-> ('Andi', 'Laptop', 950000, 1),
```

```
-> ('Budi', 'Mouse', 150000, 1);
```

```
Query OK, 2 rows affected (0.01 sec)
```

```
mysql> SELECT id, customer, node_src FROM orders;
```

```
+----+-----+-----+
```

```
| id | customer | node_src |
```

```
+----+-----+-----+
```

```
| 1 | Alice | 1 |
```

```
| 3 | Bob | 1 |
```

```
| 5 | Andi | 1 |
```

```
| 7 | Budi | 1 |
```

```
+----+-----+-----+
```

```
-- Perhatikan: semua ID dari Node1 adalah bilangan GANJIL
```

### 3.2.5 Pengujian Write di Node2

```
-- Di mm-node2
```

```
mysql> USE demo_db;
```

```
mysql> INSERT INTO orders (customer, product, amount, node_src) VALUES
```

```
-> ('Citra', 'Keyboard', 250000, 2),
```

```
-> ('Dian', 'Laptop', 950000, 2);
```

```
Query OK, 2 rows affected (0.01 sec)
```

```
mysql> SELECT id, customer, node_src FROM orders ORDER BY id;
```

```
+----+-----+-----+
```

```
| id | customer | node_src |
```

```
+----+-----+-----+
```

```
| 2 | Citra | 2 |
```

```
| 4 | Dian | 2 |
```

```
+---+-----+-----+
-- ID dari Node2 adalah bilangan GENAP
```

### 3.2.6 Verifikasi: Semua Data Terlihat di Kedua Node

```
-- Di mm-node1 (setelah insert dari node2 ter-replikasi)
mysql> SELECT id, customer, product, node_src FROM orders ORDER BY id;
+---+-----+-----+
| id | customer | product | node_src |
+---+-----+-----+
| 1 | Alice   | Laptop  | 1 |
| 2 | Citra   | Keyboard | 2 |
| 3 | Bob     | Mouse   | 1 |
| 4 | Dian    | Laptop  | 2 |
| 5 | Andi    | Laptop  | 1 |
| 7 | Budi    | Mouse   | 1 |
+---+-----+-----+
6 rows in set (0.00 sec)
```

Data dari Node1 (ID ganjil) dan Node2 (ID genap) berhasil tergabung tanpa konflik primary key di kedua node.

## 3.3 Skenario 3: Asynchronous vs Semi-Synchronous Replication

### 3.3.1 Menjalankan Mode Asynchronous

```
$ cd ~/Desktop/db-replication/sync-vs-async
$ docker compose up -d
```

```
[+] Running 4/4
✔ Container async-master Started 1.3s
✔ Container async-slave Started 2.9s
```

Setup replication dilakukan dengan cara yang sama seperti Master-Slave (langkah 3.1.3 – 3.1.5). Setelah replication aktif, saya mulai pengujian perbandingan perilaku.

### 3.3.2 Uji Async: Slave Dimatikan, Master Tetap Commit

Pada mode async, master tidak peduli apakah slave aktif atau tidak. Saya mematikan slave terlebih dahulu, kemudian melakukan INSERT di master.

```
$ docker stop async-slave
async-slave

-- Di async-master
mysql> USE demo_db;
mysql> INSERT INTO transactions (description, amount) VALUES
-> ('Transfer saat slave mati', 750000);
Query OK, 1 row affected (0.00 sec) -- ← langsung sukses, tidak ada delay!
```

INSERT berhasil seketika tanpa jeda waktu meskipun slave sedang mati. Ini menunjukkan karakteristik async: master tidak menunggu konfirmasi dari slave.



```
$ docker start async-slave
async-slave
```

```
-- Di async-slave (setelah nyala kembali)
```

```
mysql> USE demo_db;
```

```
mysql> SELECT * FROM transactions;
```

```
+---+-----+-----+-----+
| id | description          | amount | created_at    |
+---+-----+-----+-----+
| 1 | Pembayaran awal      | 100000.00 | 2026-07-14 10:01:03 |
| 2 | Transfer masuk       | 500000.00 | 2026-07-14 10:01:03 |
| 3 | Transfer saat slave mati | 750000.00 | 2026-07-14 10:15:44 |
+---+-----+-----+-----+
3 rows in set (0.00 sec)
```

Data berhasil disinkronkan setelah slave nyala kembali. Namun ada jeda waktu antara commit di master dan tersedianya data di slave (window data loss jika master crash dalam jeda ini).

### 3.3.3 Menjalankan Mode Semi-Synchronous

```
$ docker compose -f docker-compose.sync.yml up -d
```

```
[+] Running 4/4
```

```
✓ Container semisync-master Started 1.5s
```

```
✓ Container semisync-slave Started 3.2s
```

### 3.3.4 Verifikasi Plugin Semi-Sync Aktif

```
-- Di semisync-master
```

```
mysql> SHOW VARIABLES LIKE 'rpl_semi_sync%';
```

```
+-----+-----+
| Variable_name | Value |
+-----+-----+
| rpl_semi_sync_source_enabled | ON |
| rpl_semi_sync_source_timeout | 5000 |
| rpl_semi_sync_source_wait_for_replica_count | 1 |
+-----+-----+
```

```
mysql> SHOW STATUS LIKE 'Rpl_semi_sync%';
```

```
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Rpl_semi_sync_source_clients | 1 |
| Rpl_semi_sync_source_net_avg_wait_time | 1832 |
| Rpl_semi_sync_source_yes_tx | 4 |
| Rpl_semi_sync_source_no_tx | 0 |
+-----+-----+
```

Plugin aktif. Rpl\_semi\_sync\_source\_clients = 1 (slave terhubung). yes\_tx = 4, no\_tx = 0 artinya semua transaksi berhasil dikonfirmasi slave.

### 3.3.5 Uji Semi-Sync: Slave Dimatikan → Timeout Fallback

Saya mematikan slave saat semi-sync aktif untuk mengamati perilaku fallback. Sesuai konfigurasi `rpl_semi_sync_source_timeout = 5000 ms`, master akan menunggu 5 detik sebelum otomatis fallback ke mode async.

```
$ docker stop semisync-slave
semisync-slave

-- Di semisync-master
mysql> INSERT INTO transactions (description, amount) VALUES
  -> ('Test timeout semi-sync', 999999);
-- ... menunggu sekitar 5 detik ...
Query OK, 1 row affected (5.02 sec) -- ← delay 5 detik sebelum berhasil!

mysql> SHOW STATUS LIKE 'Rpl_semi_sync_source_no_tx';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Rpl_semi_sync_source_no_tx | 1 |
+-----+-----+
-- no_tx = 1 menandakan 1 transaksi yang fallback ke async karena slave tidak merespons
```

Semi-sync fallback berfungsi. Master menunggu 5 detik (timeout), kemudian commit secara async. Ini membuktikan trade-off: durability vs availability.

### 3.3.6 Perbandingan Waktu Commit

Berikut perbandingan yang saya amati antara mode Async dan Semi-Sync:

| Kondisi                   | Async (waktu commit) | Semi-Sync (waktu commit)    |
|---------------------------|----------------------|-----------------------------|
| Slave aktif & sehat       | ~0.01 detik          | ~0.03 detik                 |
| Slave lambat (lag tinggi) | ~0.01 detik          | ~0.8–1.2 detik              |
| Slave mati                | ~0.00 detik          | ~5.02 detik (lalu fallback) |

## KESIMPULAN

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, dapat disimpulkan beberapa hal berikut:

- ☐ Master-Slave Replication: Berhasil diimplementasikan dengan konfigurasi `log_bin` di master dan `read_only` di slave. Setiap perubahan data di master langsung ter-replikasi ke slave. Slave berhasil menolak operasi tulis langsung (`error super_read_only`), membuktikan fungsi proteksi bekerja.
- ☐ Master-Master Replication: Berhasil diimplementasikan dengan dua node yang saling mereplikasi. Teknik `auto_increment_offset` yang berbeda (1 dan 2) efektif mencegah konflik primary key — terbukti dari pola ID ganjil untuk Node1 dan genap untuk Node2, dan semua data terlihat lengkap di kedua sisi.
- ☐ Asynchronous vs Semi-Synchronous: Mode `async` terbukti sangat cepat namun berisiko kehilangan data saat slave mati sebelum sempat `sync`. Mode `semi-sync` menambahkan jaminan durabilitas dengan menunggu `ACK` dari slave, namun ada overhead waktu ~5 detik saat slave tidak tersedia sebelum fallback ke `async`. Pilihan mode bergantung pada kebutuhan: kecepatan (`async`) vs keamanan data (`semi-sync`).
- ☐ Docker sebagai Platform Simulasi: Penggunaan Docker Compose sangat memudahkan setup multi-node database tanpa perlu konfigurasi jaringan fisik. Setiap skenario dapat diisolasi dan diulang dengan mudah.

Secara keseluruhan, praktikum ini memberikan pemahaman yang solid tentang bagaimana database replication bekerja dalam skenario nyata, beserta trade-off yang harus dipertimbangkan saat memilih topologi dan mode replikasi yang tepat untuk suatu sistem.